Del dato al asistente: identificación de la variedad de arroz (Cammeo / Osmancik) por la forma del grano

1er Jocabed Gonzales Quispe
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e_2025100776I@uncp.edu.pe

2do Fabrizio E Saul Diaz Vega
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e_2025100774G@uncp.edu.pe

3er Vanessa Tarma Chanco
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e_2025100797D@uncp.edu.pe

4to Tomas Sebastian Vilchez Aliaga
Facultad de Ingeniería de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e_2025100802I@uncp.edu.pe

***Resumen*—El objetivo de este proyecto fue construir un chatbot que ayude a identificar si un grano de arroz pertenece a la variedad Cammeo u Osmancik a partir de siete medidas de su forma. Para ello se trabajó con el conjunto de datos de Cinar y Koklu (3 810 granos) y se desarrolló el proceso completo: limpieza y partición sin fuga de información, comparación de una línea base, una regresión logística y dos redes neuronales, elección del modelo usando solo validación, evaluación final única en prueba, versionado con fichas JSON, una API en Flask y un chatbot por reglas que la consume. Finalmente se seleccionó la regresión logística, que alcanzó una exactitud de 0.923 y un F1 macro de 0.921 en prueba; las redes neuronales no mostraron una mejora estadísticamente clara. También se analizaron las limitaciones del sistema y los cuidados necesarios para utilizarlo.**

***Index Terms*—clasificación, redes neuronales, regresión logística, API, chatbot, versionado de modelos, fuga de información**

I. Planteamiento del problema

Se utilizó el conjunto de datos *Rice (Cammeo and Osmancik)*, publicado por Cinar y Koklu [1], [2] y disponible en el repositorio UCI con el identificador 545. Contiene 3 810 granos de arroz de dos variedades y, de cada uno, siete medidas de forma obtenidas a partir de su imagen: área, perímetro, eje mayor, eje menor, excentricidad, área convexa y extensión. A partir de esas medidas se buscó distinguir entre Cammeo y Osmancik.

El sistema se planteó para un centro de acopio, donde se necesita identificar la variedad de arroz antes de registrarla. El chatbot guía al usuario para ingresar las medidas del grano y consultar el resultado. Cuando la predicción es poco confiable, recomienda solicitar ayuda a una persona.

Se consideró que los dos tipos de error no tendrían el mismo impacto. Se tomó Cammeo como clase positiva y se supuso que registrar un Cammeo como Osmancik (falso negativo) cuesta el doble que el error contrario, es decir, un costo de 2 para el falso negativo y de 1 para el falso positivo. Estos costos son valores supuestos, no precios reales, y quedaron parametrizados en `src/evaluacion.py` y puede cambiarse fácilmente. Con ese criterio, la métrica principal es el costo esperado por grano con el umbral óptimo, calculado sobre validación, siguiendo la idea habitual del aprendizaje sensible al costo [3]. Se acompaña con el F1 macro, el recall de Cammeo y el PR-AUC, porque la exactitud por sí sola puede engañar cuando las clases están repartidas en 57 % y 43 % [4].

También se definió cuándo se usaría el sistema: en la recepción, justo después de fotografiar y medir el grano. Las siete variables salen de la imagen de ese mismo grano, por lo que ya están disponibles en ese instante, y ninguna depende de información posterior ni de la etiqueta. Por ello no se identificaron variables que revelaran información futura o la etiqueta [5]. Tampoco hay atributos sensibles, ya que los datos describen granos y no personas.

El trabajo abarca la preparación de los datos, la comparación de modelos y su evaluación, además de la API y el chatbot que permite consultar las predicciones.

II. Datos y exploración

El archivo `.arff` se leyó con `scipy.io.arff`, que entrega la clase como bytes y obliga a decodificarla. Antes de entrenar los modelos se incorporaron reglas de calidad en `src/datos.py`: se revisan los valores nulos, los duplicados y la coherencia geométrica de cada grano (el eje menor no puede superar al mayor, el área convexa no puede ser menor que el área, la extensión no puede pasar de 1, la excentricidad debe ser menor que 1 y todas las medidas deben ser positivas). Los registros que fallan se eliminan y no se imputan. En los datos originales no se encontraron registros que eliminar: 0 nulos, 0 duplicados y 0 registros incoherentes, de modo que se conservaron las 3 810 filas (2 180 Osmancik y 1 630 Cammeo). También se incluyeron casos con datos defectuosos en `tests/test_datos.py` para comprobar las reglas de limpieza.

En el notebook `01_exploracion_y_modelado.ipynb` se analizaron las distribuciones y relaciones entre las

Tabla I
VERSIONES COMPARADAS Y SU CONFIGURACIÓN

Versión	Tipo	Configuración
`baseline_trivial`	Línea base	`DummyClassifier`, estrategia `prior`
`regresion_logistica`	Modelo simple	$C = 1.0$, `max_iter`=1000, variables estandarizadas
`mlp_v1`	Red 1	1 capa de 16 neuronas ReLU, L2 ($\alpha = 10^{-3}$), parada temprana (15 épocas sin mejora, 15 % interno), batch 64
`mlp_v2`	Red 2	2 capas (32-16) ReLU, L2 ($\alpha = 10^{-2}$), parada temprana, batch 64

variables. La excentricidad es una función exacta de los dos ejes (el error máximo es de 5.5×10^{-8}), por lo que la API la calcula cuando no se envía. El área, el perímetro y el área convexa tienen correlaciones de 0.97 o más entre sí (Fig. 5, Anexo A), así que los coeficientes de la regresión no deben interpretarse uno por uno. Los granos Cammeo resultan más grandes y alargados (área media de 14 163 píxeles frente a 11 550, y eje mayor de 205 frente a 176), mientras que el eje menor y la extensión separan poco las clases; en los tamaños intermedios las distribuciones se solapan (Fig. 1). Finalmente, el desbalance es leve (42.8 % de Cammeo), pero basta para que convenga considerar más métricas que la exactitud.

Para evitar que el modelo aprendiera información de validación o prueba, los datos se dividieron en 60 % para entrenamiento, 20 % para validación y 20 % para prueba (2 286, 762 y 762 filas), de forma estratificada y con semilla 42. El `StandardScaler` forma parte de un `Pipeline` y se ajusta únicamente con el conjunto de entrenamiento; una prueba automática comprueba que la media que usa coincide con la de entrenamiento y no con la de todos los datos. También se verificó que no hubiera filas repetidas entre las tres particiones (0, 0 y 0 coincidencias).

III. MODELADO Y COMPARACIÓN

Se entrenaron y compararon cuatro versiones, resumidas en la Tabla I. Primero se incluyó un clasificador trivial como punto de referencia (`DummyClassifier` con estrategia `prior`), que estima probabilidades según las frecuencias de las clases y, con el umbral predeterminado, predice la clase mayoritaria. También se probó una regresión logística como alternativa sencilla sobre variables estandarizadas. También se entrenaron dos redes neuronales para comparar sus resultados con el modelo simple: una de una capa oculta de 16 neuronas y otra de dos capas (32 y 16). Ambas usan activación ReLU, regularización L2 y parada temprana. Se implementaron con `MLPClassifier` de scikit-learn [6], que no ofrece dropout ni pesos de clase; por eso el desbalance leve se manejó con el umbral por costo y con métricas robustas. Como el entorno de desarrollo no contaba con TensorFlow ni PyTorch, las redes se definieron con scikit-learn en `src/modelo.py`.

La Tabla II presenta los resultados en validación, con el umbral óptimo de cada modelo según el costo, junto con el F1 macro de una validación cruzada estratificada 5×2 realizada solo sobre el conjunto de entrenamiento [7]. La línea base acierta 0.57 con umbral 0.5, y ese valor es el mínimo que

Tabla II
VALIDACIÓN (UMBRAL ÓPTIMO POR COSTO) Y VALIDACIÓN CRUZADA ESTRATIFICADA 5×2 SOLO SOBRE ENTRENAMIENTO (UMBRAL 0.5)

Versión	Umbral	F1 m.	Rec.	PR-AUC	Costo	CV F1 m.
`baseline_trivial`	0.05	0.300	1.000	0.428	0.5722	0.364 ± 0.000
`regresion_logistica`	0.49	0.937	0.945	0.977	0.0853	0.930 ± 0.010
`mlp_v1`	0.42	0.928	0.948	0.975	0.0932	0.925 ± 0.010
`mlp_v2`	0.35	0.926	0.957	0.976	0.0919	0.926 ± 0.009

F1 m.: F1 macro; Rec.: recall de Cammeo; Costo: costo por grano.

cualquier modelo debe superar. Las tres versiones restantes quedan muy cerca entre sí, también en las curvas ROC y de precisión-recall (Fig. 6, Anexo A). El entrenamiento de las redes se repitió con cinco semillas para medir la variación de los resultados, obteniéndose un F1 macro de 0.923 ± 0.005 para `mlp_v1` y de 0.924 ± 0.009 para `mlp_v2`. También se aplicó la prueba de McNemar [8] sobre validación, que dio $p = 0.38$ al comparar la regresión logística con `mlp_v1` y $p = 0.11$ al compararla con `mlp_v2`.

Con estos resultados no se encontró evidencia de que las redes neuronales mejoraran a la regresión logística. Las diferencias entre las tres versiones son de hasta 0.005 de F1 macro en la validación cruzada y de hasta 0.012 en la validación única, es decir, del mismo orden que la desviación entre particiones (cerca de 0.01), y McNemar no permite distinguirlas del azar. En este proyecto, la regresión logística ofreció un resultado competitivo sin añadir la complejidad de una red neuronal (Fig. 2).

IV. EVALUACIÓN Y VERSIONADO

IV-A. Elección de la versión y evaluación final

La regla de selección se definió antes de revisar los resultados: gana el modelo con menor costo esperado por grano usando el umbral óptimo de cada uno; los que quedan a 0.01 o menos del mejor se consideran empatados y se prefiere el más simple; y la línea base nunca puede promoverse. Con esa regla, la regresión logística obtuvo el mejor costo en validación (0.0853) y quedó empatada con `mlp_v1` y `mlp_v2`, por lo que se seleccionó por su menor complejidad. Su umbral, 0.49, se escogió en validación minimizando el costo (Fig. 3).

Una vez elegido el modelo, se evaluó en el conjunto de prueba una sola vez. El código registra la evaluación en `modelos/prueba_evaluada.json`. Antes de una nueva evaluación comprueba si existe ese registro y, de ser así, genera una excepción para impedir otra consulta al conjunto de prueba. Esta protección depende de conservar el archivo de registro. En esa evaluación se obtuvo una exactitud de 0.923, un F1 macro de 0.921, una precisión de Cammeo de 0.894, un recall de Cammeo de 0.929, un ROC-AUC de 0.976, un PR-AUC de 0.968 y un costo de 0.108 por grano. La matriz de confusión fue TN = 400, FP = 36, FN = 23 y TP = 303. El rendimiento en prueba es algo menor que en validación (F1 macro de 0.921 frente a 0.937), una diferencia compatible con la variabilidad del muestreo, que en la validación cruzada fue de cerca de 0.01.

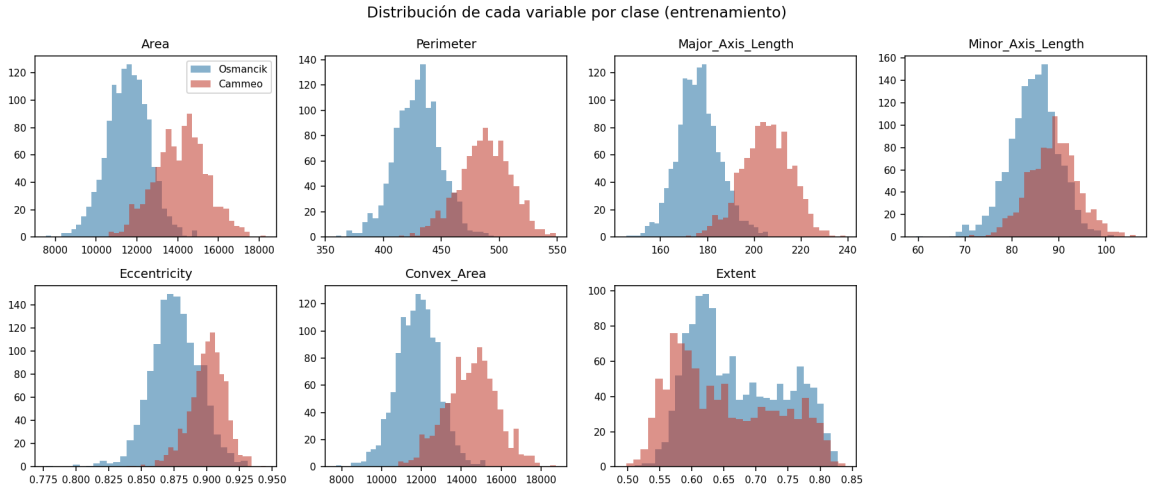


Figura 1. Distribución de cada variable por clase (conjunto de entrenamiento).

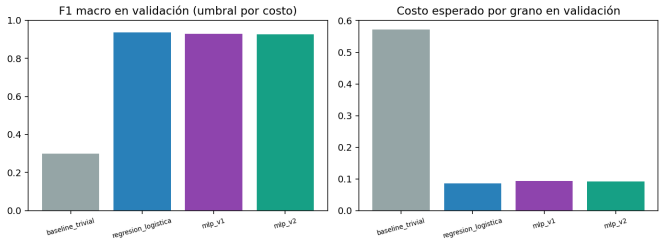


Figura 2. F1 macro y costo esperado por grano en validación.

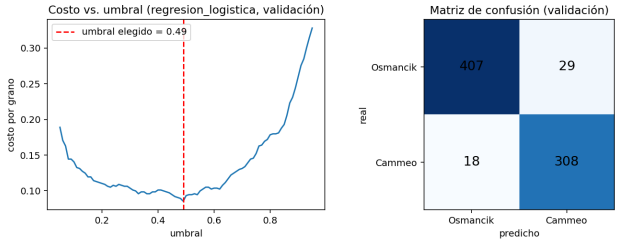


Figura 3. Costo frente al umbral y matriz de confusión de la versión elegida (validación).

IV-B. Análisis de errores

Al revisar los errores en prueba, se observó que el error no está repartido de forma pareja. Se concentra en los granos de tamaño intermedio, justo donde las dos variedades se solapan: allí la tasa de error llega a 16.6 %, frente a 3.0 % en los granos pequeños y 4.6 % en los grandes (Tabla III). Por clase, el error es de 8.3 % en Osmancik y de 7.1 % en Cammeo.

IV-C. Versionado

Cada versión entrenada se guardó en `modelos/<versión>.joblib` junto con una ficha JSON que registra la fecha (UTC), el hash SHA-256 de los datos, los resultados de calidad, los tamaños de las particiones, los parámetros, el preprocesamiento, el umbral, los costos supuestos, los rangos de entrada, las métricas de validación, la variabilidad, la semilla y las versiones de las librerías. El archivo `modelos/promovida.json` indica cuál es la versión que usa la API y el motivo de su elección. El entrenamiento utiliza semillas fijas para favorecer la reproducibilidad; la estabilidad se analiza además con validación cruzada y diferentes semillas.

Tabla III
TASA DE ERROR POR GRUPO EN EL CONJUNTO DE PRUEBA

Grupo	n	**Tasa de error**
Clase Osmancik	436	8.3 %
Clase Cammeo	326	7.1 %
Tamaño pequeño (tercil de área)	267	3.0 %
Tamaño mediano (tercil de área)	235	16.6 %
Tamaño grande (tercil de área)	260	4.6 %

V. API

La API se implementó con Flask [9] (`src/api.py`). Al arrancar, el servicio carga la versión promovida una sola vez, ya que leer y deserializar el modelo en cada solicitud sería lento e innecesario. Tiene tres endpoints: `GET /salud`, para comprobar que el servicio responde; `GET /info`, que devuelve el contrato de entrada, los rangos y el desempeño en validación; y `POST /predecir`, que recibe un JSON con las medidas del grano. Los campos `Area`, `Perimeter`, `Major_Axis_Length`, `Minor_Axis_Length`, `Convex_Area` y `Extent` son obligatorios, mientras que `Eccentricity` es opcional y se calcula con los ejes cuando falta. Si la solicitud es válida, responde con código 200 e incluye la clase predicha, su probabilidad, la probabilidad de Cammeo, el nivel de confianza (alta o baja), la versión del modelo, el umbral y la latencia en milisegundos.

La validación de entradas está concentrada en `src/preprocesamiento_api.py`. Allí se rechazan campos faltantes o desconocidos, texto no numérico (aunque acepta coma decimal), booleanos, nulos, NaN o infinito, valores menores o iguales que cero, `Extent` mayor que 1, ejes invertidos, área convexa menor que el área, excentricidad incoherente con los ejes y valores fuera del rango plausible, definido entre el 50 % del mínimo y el 150 % del máximo observados en entrenamiento. Los problemas de una misma solicitud se agrupan en una respuesta JSON con los campos `error` y `detalles`, y código HTTP 400. También se responde 400 ante un JSON mal formado, y 404 y 405 en formato JSON.

La latencia se midió con 500 solicitudes usando el cliente de pruebas de Flask en un solo núcleo: el cómputo medio fue de 1.52 ms, el percentil 95 de 2.27 ms y el máximo de 4.13 ms. Este valor se devuelve en cada respuesta, tanto en el campo `latencia_ms` como en la cabecera `X-Latencia-ms`. Cabe señalar que la medición no incluye red real ni servidor de producción. Las pruebas de `tests/test_api.py` cubren el caso exitoso, dos granos de referencia, la excentricidad opcional, los números enviados como texto, la latencia, el contrato y una batería parametrizada con cada caso de error.

VI. Chatbot

El chatbot (`src/chatbot.py`) sigue un flujo de preguntas definido y consulta la API mediante HTTP. Se optó por un sistema basado en reglas porque solo necesita recoger las medidas del grano, validar las respuestas y mostrar la predicción. Esto facilita el control de los mensajes, aunque limita la interpretación de preguntas abiertas. La Fig. 4 muestra el recorrido de la conversación y cómo se manejan los errores.

La conversación empieza con un saludo y un aviso de privacidad, y luego el chatbot pide las seis medidas una por una, cada una con una explicación en lenguaje sencillo y un ejemplo. Acepta coma decimal y texto alrededor del número (por ejemplo, «son 14 162 px»), y rechaza las respuestas con varios números, con valores negativos o con una extensión mayor que 1. Cuando no entiende, repite la pregunta e indica cuántos intentos quedan; después de tres respuestas ilegibles seguidas ofrece contactar con una persona del equipo y muestra los números de los integrantes. Una respuesta válida reinicia ese contador. Si la API rechaza un dato con un error 400, el chatbot muestra el mensaje, borra solo el dato señalado y lo vuelve a pedir; tras tres rechazos, o si la API no responde, también ofrece esos contactos. La opción de contactar con una persona aparece además cuando el usuario escribe «persona», y en cualquier momento puede usar los comandos *ayuda*, *reiniciar* y *cancelar*.

Para que cualquier usuario entienda la respuesta, el chatbot muestra la variedad estimada y la seguridad en porcentaje. Si la confianza es baja, recomienda medir de nuevo o contactar con el equipo, y en todos los casos recuerda que se trata de una estimación basada en la forma que no reemplaza al laboratorio, citando el acierto en validación que le informa la API. En cuanto a privacidad, no guarda nada: borra las medidas al terminar, reiniciar o cancelar, e ignora y avisa si el usuario escribe correos o teléfonos. A continuación se muestra un fragmento de diálogo con una respuesta inválida:

```
Asistente: (1/6) Dime el área del grano: los píxeles que
ocupa el grano en la imagen. Por ejemplo: 12000.
Usuario: no sé
Asistente: No encontré un único número en tu respuesta.
Intentemos otra vez (2 intentos más antes de pasarte con
una persona).
...
Asistente: Resultado: probablemente es arroz Cammeo
(seguridad estimada: 98 %). Recuerda: es una estimación
basada solo en la forma del grano; no reemplaza un
análisis de laboratorio.
```

Además del chat por consola, se desarrolló una interfaz web (`src/web/index.html`) servida por la API en la ruta `/`. Tiene burbujas de conversación, botones de respuesta rápida, una barra de progreso y una tarjeta de resultado con un medidor de probabilidad, y ofrece también un modo formulario con deslizadores y granos de ejemplo. Usa el mismo chatbot y la misma validación que la API, y las conversaciones viven solo en memoria.

VI-A. Calidad y pruebas automatizadas

Para revisar el funcionamiento del proyecto, se incluyeron 97 casos de prueba con `pytest` en la carpeta `tests/`. Cubren los datos (incluyendo datos defectuosos sintéticos), los modelos y la evaluación (reproducibilidad, una sola mirada a la prueba y regla de promoción), la API (el caso exitoso y 19 casos de error), el chatbot (camino feliz, caminos alternativos, errores, privacidad y API caída) y la interfaz web (sesiones independientes, progreso y reinicio). El `README.md` explica cómo ejecutar todo desde cero y `requirements.txt` fija las dependencias.

VII. Limitaciones y ética

El modelo acierta cerca del 92 % en el conjunto de prueba, lo que significa que se equivoca en aproximadamente 1 de cada 13 granos, y mucho más en los tamaños intermedios, donde el error llega a 17 %. Por eso se presenta como una herramienta de apoyo, no como una decisión definitiva. Otra limitación importante es que no se probó su capacidad de generalizar: se usó un único conjunto de imágenes, con una cámara y condiciones de captura fijas, y solo dos variedades. Como las medidas están en píxeles, un grano de otra variedad o medido con otra cámara o resolución podría clasificarse con una seguridad que no corresponde.

También se reconocen algunas decisiones y limitaciones de la evaluación. La razón de costos de 2 a 1 entre los errores es un supuesto del escenario y no se basa en precios reales; con otros costos el umbral cambiaría. Además, se trabajó con una sola partición de prueba de 762 granos, de modo que la incertidumbre de la exactitud es de aproximadamente ± 2 puntos. Tampoco se encontró que las redes neuronales superaran a la regresión logística; con otros datos o más variables el resultado podría ser distinto, y no fue posible usar dropout ni pesos de clase por trabajar con scikit-learn. Por último, las probabilidades que entrega el modelo no están calibradas de forma explícita: en la curva de calibración sobre

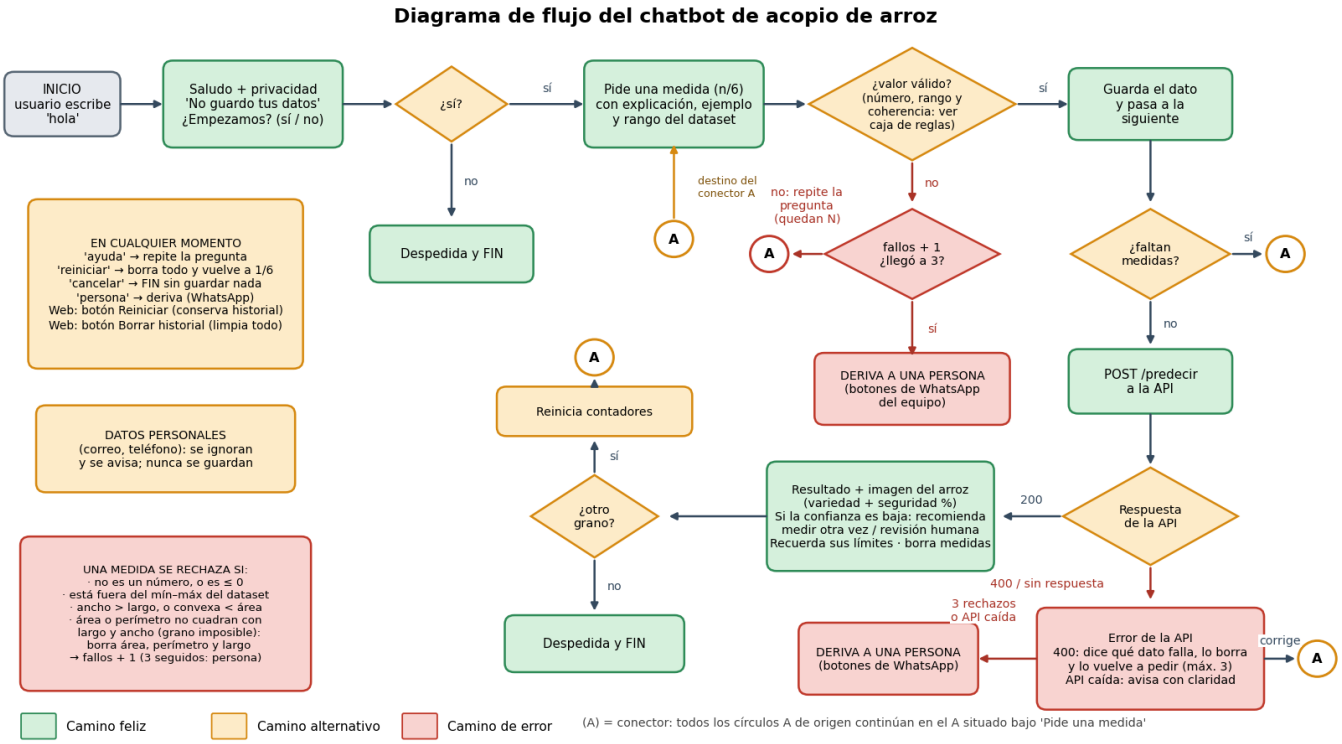


Figura 4. Diagrama de flujo del chatbot (caminos feliz, alternativo y de error).

validación (Fig. 7, Anexo A) se observan desviaciones respecto de la diagonal en la zona intermedia, por lo que el porcentaje de seguridad que muestra el chatbot debe leerse como una referencia y no como una probabilidad exacta.

El riesgo relacionado con datos personales se considera bajo, porque los datos no incluyen información personal ni atributos sensibles. Aun así, se identificaron tres riesgos propios del uso del sistema: la sobreconfianza en una estimación que a veces falla, su empleo en decisiones económicas a pesar de haber sido entrenado con un solo conjunto de imágenes, y el error desigual según el tamaño del grano. Para reducir estos riesgos, se incorporó la opción de contactar con integrantes del equipo cuando la confianza es baja, se audita el error por tamaño y por clase (Sección IV), y el chatbot declara sus límites y no guarda datos. Como trabajo futuro queda validar el sistema con granos medidos en un centro de acopio real, ampliarlo a más variedades, detectar entradas fuera de la distribución de entrenamiento y recalibrar las probabilidades.

VIII. Contribución de cada integrante

La Tabla IV resume las principales contribuciones de cada integrante al desarrollo del proyecto.

Tabla IV
Contribución de cada integrante

Integrante	Contribución principal
J. Gonzales	Carga, limpieza y partición de datos; análisis exploratorio y dependencias iniciales.
F. Diaz	Modelos, entrenamiento, evaluación, versionado, archivos de resultados y documentación.
V. Tarma	Figuras de análisis y resultados; elaboración del diagrama de flujo del chatbot.
S. Vilchez	Desarrollo y mejoras del chatbot, integración con la API, validación de entradas y pruebas automatizadas.

IX. Declaración de uso de herramientas de IA

Durante el desarrollo se utilizó Claude (Anthropic) para elaborar y ajustar código del proyecto, especialmente el chatbot y su integración con la API. Los resultados del entrenamiento se contrastaron con los archivos JSON generados por el proyecto. También se revisaron la separación de los datos, el criterio de selección del modelo y el registro que controla la evaluación final. Las sugerencias de la herramienta se tomaron como referencia y se revisaron antes de incorporarlas al trabajo.

Referencias

[1] I. Cinar and M. Koklu, "Classification of rice varieties using artificial intelligence methods," *Int. J. Intell. Syst. Appl. Eng.*, vol. 7, no. 3, pp. 188–194, 2019.

[2] I. Cinar and M. Koklu, "Rice (Cammeo and Osmancik)," UCI Machine Learning Repository, 2019. [En línea]. Disponible: https://archive.ics.uci.edu/dataset/545

[3] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2001, pp. 973–978.

[4] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
[5] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, art. 15, 2012.
[6] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[7] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.
[8] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[9] Pallets Projects, “Flask documentation.” [En línea]. Disponible: https://flask.palletsprojects.com

APÉNDICE
FIGURAS COMPLEMENTARIAS

Este anexo reúne figuras generadas por el mismo código del proyecto que apoyan lo expuesto en el texto principal.

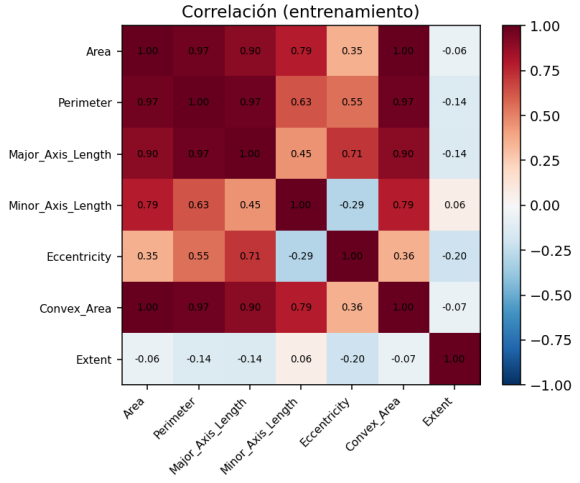


Figura 5. Matriz de correlación de las siete variables (conjunto de entrenamiento). Area, Perimeter y Convex_Area están fuertemente correlacionadas entre sí.

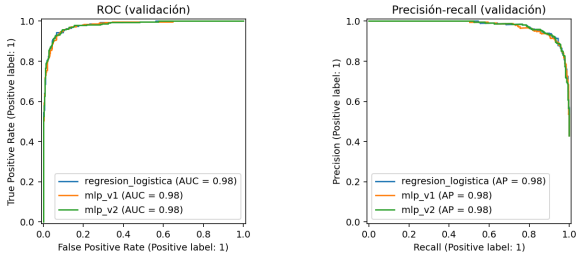


Figura 6. Curvas ROC y de precisión-recall de la regresión logística y las dos redes en validación. Las tres versiones prácticamente se superponen.

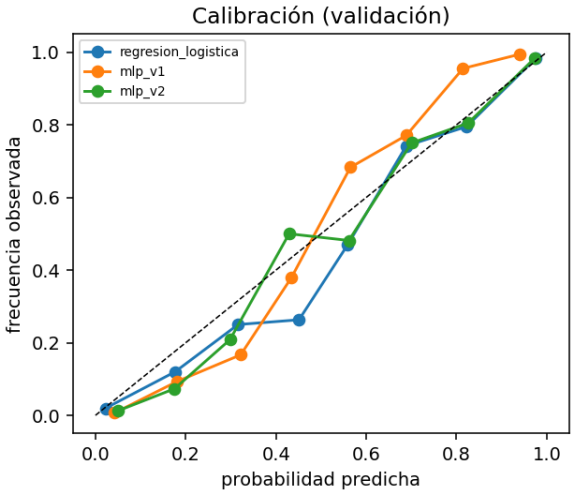


Figura 7. Curva de calibración de las tres versiones en validación. La línea punteada representa la calibración perfecta.